Parsek — Logistics Routes Design

Design specification for Parsek's automated cargo delivery system — covering route creation, dispatch scheduling, resource/inventory/crew transfer between unloaded vessels, endpoint resolution, transfer window computation, and round-trip linking across the rewind timeline.

Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to a timeline, rewind to earlier points, and see previously recorded missions play back as ghost vessels alongside new ones. This document specifies how committed recording chains are turned into logistics routes that physically move cargo (resources, inventory items, crew) between vessels.

Version: 0.2 (updated to reflect Phase 11/12 planning decisions) **Prerequisite:** Phase 11 (Resource Snapshots) must be implemented before routes. Phase 10 (Location Context) is already complete. **Out of scope:** Ghost playback engine, recording system, chain structure, game actions recalculation engine. See `parsek-flight-recorder-design.md` and `parsek-game-actions-and-resources-recorder-design.md` for those.

---

1. Introduction

This document specifies how Parsek turns committed recording chains into automated supply routes. Routes are chain-sequential — the player flies a cargo mission (potentially multi-stop), commits it, and Parsek replays the entire chain in sequence each dispatch cycle with cargo delivery at each stop. It covers:

- The "Start/End Route Recording" workflow and route analysis engine
- How stops and delivery manifests are derived from the committed chain
- Chain-sequential playback: segments play in order, delivery between segments, restart after dispatch interval
- The dispatch cycle: check timing, check capacity, check origin resources, deduct, deliver
- Endpoint resolution: surface proximity fallback vs orbital PID matching
- Transfer window scheduling via synodic period computation
- Round-trip linking for paired one-way routes
- Cargo modification on unloaded vessels (resources via ProtoPartResourceSnapshot, inventory via ModuleInventoryPart, crew via ProtoCrewMember)
- Timeline integration with epoch isolation for revert safety
- Module architecture: self-contained `Logistics/` directory with 4 integration seams
- Edge cases: destruction, full tanks, competing routes, time warp, reverts

1.1 What happens when the player creates a route

The player starts a route recording session by clicking "Start Route Recording" in the Parsek UI. They then fly a cargo mission manually — drive a rover with fuel to a base, launch a tanker to an orbital station, or fly a multi-stop supply run. During the flight, they dock at each destination, transfer cargo (resources, inventory items, crew) using KSP's standard UI, and undock. Parsek records the whole chain normally.

When done, the player clicks "End Route Recording." Parsek's route analysis engine walks the committed chain to extract stops — each dock-transfer-undock sequence becomes a route stop with its own delivery manifest. The player sees a route summary (origin, stops, deliveries, transit time), sets the dispatch interval, and confirms. The route goes live.

1.2 What happens each cycle

On schedule, the route scheduler begins chain-sequential playback. It tells the ghost playback engine to play segment 1 of the recording chain. When segment 1 completes, the scheduler starts segment 2 — and if a stop falls between those segments, delivery happens at that stop. This continues through the entire chain. When the last

segment finishes, the cycle is complete: final delivery triggers, and the scheduler waits for the dispatch interval before restarting from segment 1.

Before each cycle, the route evaluates whether dispatch is possible. It checks the destination (is there room?), the origin (are there enough resources?), and whether the transit window is open. If everything checks out, origin resources are deducted. The ghost replays the recorded chain visually during transit.

1.3 What the player sees

Situation	What happens
Start Route Recording	Player clicks button, flies mission normally. Parsek records the chain.
End Route Recording	Route analysis extracts stops from committed chain. Player sees summary, sets interval, confirms.
Route dispatches on schedule	Origin resources deducted (if non-KSC). Ghost begins chain-sequential replay.
Route delivers at each stop	Cargo (resources, inventory, crew) appears at stop vessel as ghost reaches each stop.
Destination tanks full	Cycle skipped. Origin NOT deducted. No ghost replay for skipped cycle.
Origin runs out of resources	Dispatch delayed until resources available. Route resumes automatically.
Destination destroyed (surface)	Proximity fallback auto-reconnects to rebuilt base at same location.
Destination destroyed (orbital)	Route halted (`EndpointLost`). Player must re-target to new station.
Player reverts past a dispatch	Epoch isolation invalidates dispatch. Origin resources restored.
Two routes linked as round-trip	They alternate: Route A completes → Route B dispatches → B completes → A dispatches.

1.4 Example: fuel delivery rover

```
ROUTE RECORDING SESSION:
  Player clicks "Start Route Recording"
  Segment 1: Rover departs KSC runway with 200 LF
  Segment 2: Rover docks at base (dock-segment EndResources: 195 LF)
  Segment 3: Player transfers 150 LF to base (undock-segment StartResources: 45 LF)
  Segment 4: Rover undocks, drives back to KSC
  Player clicks "End Route Recording"

ROUTE ANALYSIS PRODUCES:
  Origin:     KSC runway area (free -- no deduction)
  Stop 1:     base location (from DockTargetVesselPid + dock coordinates)
  Delivery:   150 LF per cycle (195 - 45 from dock/undock boundary resources)
  Transit:    chain duration (~10 min)
```

```
  Interval:   player-set (minimum = chain duration)

EACH CYCLE (chain-sequential):
  UT=0:     Dispatch. Ghost rover starts segment 1.
  UT=seg1:  Segment 1 completes. Stop 1 delivery: 150 LF added to base.
            Ghost starts segment 2 (return trip).
  UT=total: Chain complete. Cycle done. Wait for dispatch interval.
            If base full at dispatch check -> 0 LF added, origin not deducted.
```

1.5 Example: Minmus mining supply chain

```
ROUTE A: KSC → Minmus Base (fuel supply)
  Origin:     KSC (free)
  Delivery:   800 LF + 978 Ox per cycle
  Result:     mining base stays fueled for drill + ISRU operation

ROUTE B: Minmus Base → Kerbin Depot (ore delivery)
  Origin:     Minmus base (non-KSC — must have resources)
  Cost:       1200 Ore + 600 LF + 733 Ox per cycle (start manifest)
  Delivery:   1200 Ore per cycle
  Gate:       dispatch delayed until base has all required resources

CHAIN BEHAVIOR:
  Route A delivers fuel → base mines ore → Route B ships ore to depot.
  If Route A stops → base runs dry → Route B pauses indefinitely.
  If depot tanks full → Route B skips cycles, base accumulates ore.
```

---

2. Design Philosophy

These principles govern every design decision in the logistics system. They are listed here because they inform every section that follows.

2.1 Realism and fidelity

1. **Fly it once, automate it forever.** Routes replicate exactly what the player did during the recording. The delivery amount, transit duration, and fuel cost are all derived from the real flight — not configured abstractly.

2. **Transit takes real time.** The route duration equals the recording duration. A 3-year Eeloo transfer takes 3 years per cycle. A 10-minute rover drive takes 10 minutes. No shortcuts.

3. **Transfer windows are respected.** Inter-body routes dispatch at the synodic period of the origin and destination bodies. Same-body routes can dispatch at any time but not faster than the original recording.

2.2 Resource safety

4. **No infinite cargo glitches.** Routes deliver exactly what was transferred during the recording — no more, no less. This applies to resources, inventory items, and crew equally. KSC origins are free (KSP charges funds to build vessels). Non-KSC origins deduct the transport's full start manifest. Recovery funds from the real vessel are one-time.

5. **Don't waste origin resources.** Origin is only deducted if at least one delivery resource can be accepted at the destination. If destination is completely full, the cycle is skipped and origin pays nothing. Per-resource delivery is independent: each resource fills what fits.

6. **Dock + transfer + undock required.** A route can only be created from a recording chain where the transport docked, transferred resources, AND undocked (freeing the port for the next cycle) at least once. No undock = no route. The "Start/End Route Recording" workflow makes this explicit.

2.3 Abstraction model

7. **No physical vessel during transit.** Route execution is pure math — deduct at origin, wait, add at destination. The ghost is visual only. No physics, no collisions, no orbit propagation.

8. **Endpoints are locations, not specific vessels.** Surface endpoints use vessel PID as primary match, with 50m coordinate fallback if the vessel is gone. Orbital endpoints use PID only. This survives base rebuilding, vessel replacement, and mod compatibility (transfer tubes, claws, etc.).

9. **The system doesn't produce cargo.** Parsek moves resources, inventory items, and crew between locations. Mining, ISRU, and solar power are the player's responsibility. Routes chain naturally: the output of one feeds the input of another.

2.4 Timeline integration

10. **Dispatches and deliveries are timeline events.** Route activity participates in the same ledger and epoch isolation system as game actions. Reverts invalidate dispatches from abandoned timelines. Resources are restored via quicksave.

11. **Routes persist across scenes and save/load.** All route state is serialized in the .sfs. The scheduler runs in all scenes via ParsekScenario.

---

3. Terminology

Route — a separate entity that defines a repeating resource transfer across one or more stops. Created from a committed recording chain via the "Start/End Route Recording" workflow. Uses chain-sequential ghost playback (not the per-recording loop system).

Route stop — a location where the transport docks, transfers resources, and undocks during the recorded chain. Each stop has its own endpoint and delivery manifest. A single-stop route is the common case; multi-stop routes are supported.

Recording chain — the ordered list of recordings committed during a route recording session. The route scheduler replays these segments in sequence each cycle.

Endpoint — origin or stop location of a route. Defined by body, coordinates, and the vessel PID from the dock event. Surface endpoints fall back to 50m proximity if the PID vessel is gone. Orbital endpoints use PID only.

Delivery manifest — the per-resource amounts, inventory items, and crew transferred at a stop. Resource deltas computed from dock-segment EndResources minus undock-segment StartResources (positive = delivered to station, negative = picked up from station). Inventory and crew deltas computed from the same boundary snapshots.

Cost manifest — the transport vessel's resources at recording start. For non-KSC origins, this is deducted from the origin each cycle (cargo + transit fuel).

Dispatch — the moment a route cycle begins. Origin resources are checked and deducted. A timeline event is created.

Delivery — the moment delivery occurs at a stop (between chain segments). Resources are added to or removed from the stop vessel. A timeline event is created.

Round-trip link — a scheduling constraint pairing two one-way routes: "don't dispatch me until my partner completes."

Route analysis engine — pure logic that walks the committed recording chain to extract stops, delivery manifests, and transit durations from the recording data (StartResources, EndResources, DockTargetVesselPid, location context).

---

4. Data Model

4.1 ResourceAmount

```
internal struct ResourceAmount
{
    public double amount;
    public double maxAmount;
}
```

4.2 RouteEndpoint

```
internal struct RouteEndpoint
{
    public string bodyName;        // e.g., "Kerbin", "Mun"
    public double latitude;        // from dock event trajectory point
    public double longitude;
    public double altitude;
    public uint vesselPid;         // PID of vessel docked to during recording
    public bool isOrbital;         // true = no surface proximity fallback
}
```

4.3 RouteStatus

```
internal enum RouteStatus
{
    Active,             // dispatching on schedule
    InTransit,          // dispatched, waiting for transit duration to elapse
    WaitingForResources,// origin exists but lacks resources — delayed
    DestinationFull,    // destination can't accept delivery — skipping cycles
    EndpointLost,       // destination/origin vessel gone (orbital PID miss or no surface
vessels)
    Paused              // player manually paused
}
```

4.4 RouteStop

```
internal class RouteStop
{
    public RouteEndpoint Endpoint;                          // where this stop is
    public Dictionary<string, double> DeliveryManifest;     // per-resource amounts
(positive = deliver, negative = pick up)
    public int SegmentIndexBefore;                          // chain segment index before
this stop
}
```

A single-stop route (the common case) has one RouteStop. Multi-stop routes have one RouteStop per dock-transfer-undock sequence found by the route analysis engine.

4.5 Route

```
internal class Route
{
    // Identity
    public string Id;                    // unique route ID (GUID)
    public List<string> RecordingIds;    // ordered chain of source recording IDs
    public string Name;                  // player-visible name (editable)

    // Endpoints
    public RouteEndpoint Origin;
    public List<RouteStop> Stops;        // ordered stops along the route
    public bool IsKscOrigin;             // true = no origin deduction

    // Resource transfer
    public Dictionary<string, double> CostManifest;      // per-resource origin cost (start
manifest)

    // Timing
    public double TransitDuration;       // seconds (= total chain duration)
    public double DispatchInterval;      // seconds between cycle starts
    public double NextDispatchUT;        // UT of next scheduled dispatch
    public int CurrentSegmentIndex;      // which chain segment is currently playing (0 =
not in transit)

    // Per-stop pending delivery (computed at each stop boundary during transit)
    public double? PendingDeliveryUT;    // UT when current segment completes (null if not
in transit)
    public Dictionary<string, double> PendingDeliveryAmounts; // actual amounts for current
stop (survives save/load)

    // Linking
    public string LinkedRouteId;         // paired route for round-trip (null if standalone)

    // State
    public RouteStatus Status;
    public int CompletedCycles;          // total successful cycle completions
```

```
    public int SkippedCycles;            // cycles skipped (destination full, origin empty)
}
```

Backward compatibility with single-stop routes: A single-stop route is a Route with `Stops.Count == 1` and `RecordingIds.Count == 1`. The data model handles both cases uniformly — no special-case code paths.

4.6 Serialization format

```
ROUTE
{
    id = <guid>
    name = Mun Fuel Run

    RECORDING_IDS
    {
        id = <recording-guid-1>
        id = <recording-guid-2>
        id = <recording-guid-3>
    }

    ORIGIN
    {
        bodyName = Kerbin
        latitude = -0.0972
        longitude = -74.5577
        altitude = 75.2
        vesselPid = 12345
        isOrbital = False
    }

    STOP
    {
        ENDPOINT
        {
            bodyName = Mun
            latitude = 3.2001
            longitude = -45.1234
            altitude = 612.5
            vesselPid = 67890
            isOrbital = False
        }
        segmentIndexBefore = 1
        DELIVERY_MANIFEST
        {
            LiquidFuel = 150.0
            Oxidizer = 183.3
        }
    }
    STOP
    {
        ENDPOINT
```

```
        {
            bodyName = Mun
            latitude = 5.1002
            longitude = -40.5678
            altitude = 580.0
            vesselPid = 11111
            isOrbital = False
        }
        segmentIndexBefore = 3
        DELIVERY_MANIFEST
        {
            Ore = -1200.0
        }
    }

    isKscOrigin = True
    transitDuration = 12345.6
    dispatchInterval = 43200.0
    nextDispatchUT = 55000.0
    currentSegmentIndex = 0
    pendingDeliveryUT = -1
    linkedRouteId =
    status = Active
    completedCycles = 5
    skippedCycles = 1

    COST_MANIFEST
    {
        LiquidFuel = 200.0
        Oxidizer = 244.0
    }
    PENDING_DELIVERY
    {
        // present only when Status == InTransit
        LiquidFuel = 100.0
        Oxidizer = 0.0
    }
}
```

Routes are stored in their own `ROUTES` ConfigNode section inside ParsekScenario's save data, alongside recordings and game actions. Additive — saves without routes load fine. Routes reference recordings by ID but are independent entities.

4.7 Recording extensions (Phase 11)

Phase 11 adds three manifest types to recordings. All are captured at segment boundaries (recording start, end, dock/undock events) from the vessel snapshot that already exists at those points.

Resource manifests (v1 — Phase 11):

```
public Dictionary<string, ResourceAmount> StartResources;  // manifest at recording start
public Dictionary<string, ResourceAmount> EndResources;     // manifest at recording end
```

`ResourceAmount` is a struct with `amount` and `maxAmount` fields. Resources are summed across all parts. ElectricCharge and IntakeAir are excluded (environmental noise). Extracted by `VesselSpawner.ExtractResourceManifest(ConfigNode vesselSnapshot)`.

Serialized as:

```
RESOURCE_MANIFEST
{
    RESOURCE { name = LiquidFuel  startAmount = 3600  startMax = 3600  endAmount = 200  endMax
= 3600 }
    RESOURCE { name = Oxidizer    startAmount = 4400  startMax = 4400  endAmount = 244  endMax
= 4400 }
}
```

Dock target vessel PID (v1 — Phase 11):

```
public uint DockTargetVesselPid;  // PID of vessel docked to at this segment's boundary (0 =
not a dock segment)
```

Captured during `CommitDockUndockSegment` when the event type is `Docked`. Identifies which station/base the transport docked to — the route analysis engine uses this to determine stop endpoints.

Inventory manifests (implemented — Phase 11):

KSP 1.12 `ModuleInventoryPart` items (stored parts in cargo containers). `ExtractInventoryManifest` walks MODULE > STOREDPARTS > STOREDPART nodes. `InventoryItem { count, slotsTaken }` struct + vessel-level `totalInventorySlots`. Same capture sites as resources. Enables automated parts delivery (spare wheels, solar panels, etc.).

Crew manifests (implemented — Phase 11):

Crew composition by trait (Pilot/Scientist/Engineer/Tourist). Route delivery uses generic kerbals (trait + level, not named individuals — separate from the crew reservation system). Same capture pattern. Enables automated crew rotation routes.

All manifest types are additive — missing node = no data. No format version bump.

---

5. Route Creation

5.1 Route recording workflow

Route creation uses an explicit recording session, not automatic derivation from any committed recording.

Player flow:

1. Player clicks "Start Route Recording" in Parsek UI
2. Player flies mission normally — launch, transit, dock at destination(s), transfer resources via KSP UI, undock, optionally continue to more stops, fly home
3. Player clicks "End Route Recording"
4. Parsek's route analysis engine walks the committed chain, presents route summary
5. Player sets dispatch interval, confirms
6. Route goes live

What "Start Route Recording" does:

- Sets `IsRecordingRoute = true` on `ParsekScenario` (serialized, survives scene changes)
- UI shows "Route Recording Active" indicator
- Recording behavior is unchanged — same FlightRecorder, same chain boundaries, same snapshots

What "End Route Recording" does:

- Sets `IsRecordingRoute = false`
- Triggers `RouteAnalysisEngine.AnalyzeChain(recordings)` on the committed chain
- Shows route confirmation UI with summary of derived stops

5.2 Route analysis engine

`RouteAnalysisEngine` (in `Logistics/`) is pure logic over committed Recording fields. Fully testable without KSP.

Input: ordered list of recordings committed during the route session (tagged by a shared RouteSessionId or by UT range).

Algorithm:

```
Walk chronologically through the chain:
  For each recording with DockTargetVesselPid != 0 (a dock event):
    Find the matching undock recording (next segment with Undocked event)
    stop.endpoint = {
        body, lat, lon from dock-time trajectory point,
        vesselPid = DockTargetVesselPid,
        isOrbital = altitude > body atmosphere height
    }
    stop.deliveryManifest = dock-segment EndResources - undock-segment StartResources
                            (per resource: positive = delivered to station,
                             negative = picked up from station)
    stop.transitDuration = dock UT - previous stop UT (or recording start UT)

  origin = first recording's start location + body
  isRoundTrip = last recording ends within 500m of origin (same body)
  totalTransit = last recording EndUT - first recording StartUT
```

Derived values:

- **Origin** = first recording's start location (body + coordinates, from Phase 10 location context)
- **Stops** = one RouteStop per dock-transfer-undock sequence found in the chain
- **Origin vessel PID** = vessel PID at recording start
- **Stop vessel PIDs** = `DockTargetVesselPid` from each dock segment (Phase 11)
- **IsKscOrigin** = true if origin body is Kerbin and coordinates are near a launch site
- **DeliveryManifest per stop** = dock-segment EndResources minus undock-segment StartResources (per resource)
- **CostManifest** = StartResources on the first recording (transport vessel start state)
- **TransitDuration** = total chain duration (last recording EndUT minus first recording StartUT)
- **DispatchInterval** = TransitDuration (default; player can increase). For inter-body routes: defaults to synodic period of origin and destination bodies.
- **RecordingIds** = ordered list of all recording IDs in the chain

5.3 Validation

The route analysis pass validates the chain:

1. At least one dock event exists (recording with `DockTargetVesselPid != 0`)
2. At least one undock event exists AFTER the dock
3. At least one resource, inventory item, or crew member changed on the transport between the dock and undock snapshots

If validation fails, the route confirmation UI shows what's missing (e.g., "Transport must undock from destination to enable route — docking port needs to be free for the next cycle").

5.4 Player confirmation

Route configuration panel shows derived values: origin, stops with delivery manifests, total transit time, cost manifest. Player can edit name, dispatch interval, and enable/disable. On confirm, route is created and scheduling begins.

5.5 Multi-stop routes

The route analysis engine naturally handles multi-stop chains. Each dock-transfer-undock sequence in the committed chain becomes a separate RouteStop. Delivery at each stop is independent.

Multi-stop example:

```
Player flies: KSC -> Base A (dock, deliver 150 LF) -> Base B (dock, pick up 1200 Ore) -> KSC

Route analysis produces:
  Origin: KSC, Kerbin
  Stop 1: Base A on Mun -- delivers 150 LF, 183 Ox
  Stop 2: Base B on Mun -- picks up 1200 Ore
  Total transit: 2d 4h
  Round-trip: Yes
```

---

6. Dispatch and Delivery

6.1 Dispatch evaluation

Trigger: The route scheduler (`RouteScheduler`) runs each physics frame (or once per second during warp) in all scenes via `RouteOrchestrator`, called from `ParsekScenario.Update`.

For each route with Status in {Active, WaitingForResources, DestinationFull} and `NextDispatchUT <= currentUT`:

Routes with Status InTransit, Paused, or EndpointLost are excluded from dispatch evaluation.

Step 1: Check round-trip link. If `LinkedRouteId` is set and the linked route has `Status == InTransit`, skip — wait for partner to complete.

Step 2: Check first stop destination. Find vessels at the first stop's endpoint (section 7). If NO vessels found at all -> set `Status = EndpointLost`, skip cycle. If vessels found but zero capacity for all delivery resources -> set `Status = DestinationFull`, increment `SkippedCycles`, advance `NextDispatchUT`. If capacity available -> proceed.

Step 3: Check origin (if non-KSC). Find vessels at origin endpoint (section 7). If no vessels found at all -> set `Status = EndpointLost`, skip cycle. If vessels exist but insufficient resources for CostManifest -> set `Status = WaitingForResources`, do NOT advance `NextDispatchUT` (re-check next frame). Note: only one dispatch per route per evaluation cycle — no catch-up storm.

Step 4: Dispatch. Deduct full CostManifest from origin (transit fuel burned regardless of partial delivery). Set `CurrentSegmentIndex = 0`. Tell the ghost playback engine to play the first recording in `RecordingIds`. Set `Status = InTransit`. Create ROUTE_DISPATCHED timeline event. Advance `NextDispatchUT`.

6.2 Chain-sequential playback

Integration hook: `OnPlaybackCompleted` (not `OnLoopRestarted`). The route scheduler listens for segment completion events from the ghost playback engine via `RouteOrchestrator.OnSegmentCompleted(evt)`.

When segment N completes:

1. **Check for stop delivery.** If a RouteStop has `SegmentIndexBefore == N`, execute delivery at that stop (see section 6.3).
2. **Advance to next segment.** If `CurrentSegmentIndex < RecordingIds.Count - 1`: increment `CurrentSegmentIndex`, tell the playback engine to play the next recording in the chain.
3. **Cycle complete.** If `CurrentSegmentIndex` was the last segment: execute any final stop delivery, set `Status = Active`, increment `CompletedCycles`, reset `CurrentSegmentIndex = 0`. The scheduler waits for the dispatch interval before restarting from segment 0.

Routes do NOT use the per-recording loop toggle. The route scheduler owns all timing and sequencing. Routes and loops are siblings — both use the ghost playback engine, but routes chain multiple trajectories sequentially with delivery logic between segments.

6.3 Per-stop delivery execution

Trigger: A chain segment completes and a RouteStop falls at that boundary.

1. **Find stop vessels** at the stop's endpoint (section 7). If NO vessels found -> log warning. Resources for this stop are lost (transit already underway). Continue to next segment.
2. **For each resource in the stop's `DeliveryManifest`:** distribute across stop vessel tanks, clamped to current `maxAmount`. Positive values add resources (delivery). Negative values remove resources (pickup — deducted from stop vessels). For unloaded vessels: modify `ProtoPartResourceSnapshot.amount` directly, respect `flowState`. For loaded vessels: use `Part.RequestResource()`.
3. **Create ROUTE_DELIVERED timeline event.** Record amounts actually delivered and the stop index.
4. **Deliver inventory and crew** from their respective manifests. Inventory items are placed into destination cargo slots; crew (generic kerbals by trait) are assigned to available seats. See sections 4.7 for manifest formats.

6.4 Legacy single-delivery execution

For backward compatibility and simple single-stop routes, the delivery logic described in section 6.3 applies identically — a single-stop route has one RouteStop, and delivery executes once when the segment before that stop completes.

6.5 Per-resource independent delivery

Each delivery resource at each stop is evaluated independently:

```
For each resource in stop.DeliveryManifest:
    capacity = sum of (maxAmount - amount) across all stop vessel tanks for this resource
    deliver = min(manifest amount, capacity)
    actualDelivery[resource] = deliver
```

Origin cost: deduct full CostManifest at dispatch time if ANY delivery resource across ANY stop has a non-zero amount. Zero deduction only if total delivery is zero across all resources at all stops.

6.6 Pause, unpause, and re-target

Pause: Player clicks Pause in route UI → `Status = Paused`. Route excluded from dispatch evaluation. If in transit, the current chain-sequential playback may continue visually (ghost keeps moving), but no further deliveries or segment advances occur.

Unpause: Player clicks Resume → `Status = Active`. Route re-enters dispatch evaluation on next scheduler tick. `NextDispatchUT` is recalculated if stale (advanced to next valid dispatch time from currentUT).

Re-target (EndpointLost recovery): Player selects a new destination vessel in the route UI → endpoint coordinates and vesselPid updated, `Status` transitions from `EndpointLost` to `Active`. Same mechanism for origin re-targeting on non-KSC routes.

---

7. Endpoint Resolution

7.1 Algorithm

```
ResolveEndpointVessels(endpoint):
    1. Find vessel by endpoint.vesselPid in FlightGlobals.Vessels
       → if found: return [vessel]
    2. If not found AND endpoint.isOrbital:
       → return [] (orbital endpoints have no fallback)
    3. If not found AND endpoint is surface:
       → scan FlightGlobals.Vessels for all vessels within 50m
         of (body, lat, lon, alt)
       → return matches with available tank capacity
```

7.2 Surface vs orbital behavior

- **Surface endpoints:** PID primary, 50m proximity fallback. Handles base rebuilding — a new vessel at the same spot auto-becomes the endpoint.
- **Orbital endpoints:** PID only. Orbital coordinates change every second, so proximity fallback does not work. If an orbital station is destroyed and rebuilt, the player must re-target the route.

7.3 Loaded vs unloaded vessels

If the endpoint vessel is loaded (player is within physics range), use `Part.RequestResource()` for resource operations. If unloaded, use `ProtoPartResourceSnapshot.amount` directly. Both paths apply to origin (deduction) and destination (delivery).

---

8. Transfer Window Scheduling

8.1 Synodic period computation

```
SynodicPeriod(originBody, destBody):
    if originBody == destBody:
        return 0  // same body, no transfer window
    // Walk up to common parent
```

```
    a = originBody, b = destBody
    while a.referenceBody != b.referenceBody:
        if a hierarchy depth > b: a = a.referenceBody
        else: b = b.referenceBody
        if a is Sun or b is Sun: return 0
    // a and b now orbit the same parent
    T1 = a.orbit.period, T2 = b.orbit.period
    if T1 == T2: return 0
    return abs(1 / (1/T1 - 1/T2))
```

Handles cross-system routes: Mun→Laythe walks up to Kerbin/Jool orbiting Sun. Guards against Sun-orbiting bodies and equal-period edge cases.

8.2 Dispatch interval rules

- **Same-body routes:** Player-set interval. Minimum = recording duration.
- **Inter-body routes:** Default = synodic period. Player can override.
- **Gravity assist routes:** Two-body synodic approximation (intermediate flybys not tracked). Player can fine-tune.

---

9. Round-Trip Linking

9.1 How round trips work

A round trip is two separate one-way routes, each from its own recording:

1. Player flies tanker KSC → Station. Parsek spawns real tanker at station after first playback.
2. Player flies spawned tanker Station → KSC. Second recording created.
3. Both recordings become routes. Player links them as a round-trip pair.

9.2 Scheduling constraint

Linked routes alternate: **don't dispatch me until my partner completes.**

```
Route A completes → Route B dispatches → Route B completes → Route A dispatches → ...
```

9.3 Implementation

Player selects two routes in the UI and clicks "Link as Round Trip." Sets `LinkedRouteId` on both. The dispatch evaluation (section 6.1, step 1) checks whether the partner is `InTransit`. Unlinking clears `LinkedRouteId` on both. Pausing a partner does NOT block the other (linked-wait only applies to `InTransit`).

---

10. Edge Cases

10.1 Destination destroyed, surface base

Scenario: Mun base destroyed. Player rebuilds at same spot. **Behavior:** PID match fails. Surface proximity fallback finds new vessel within 50m. Route auto-reconnects. **v1 limitation:** If rebuilt >50m from original, player must create a new route.

10.2 Destination destroyed, orbital station

Scenario: Kerbin station destroyed. Player rebuilds. **Behavior:** PID match fails. No proximity fallback. `Status = EndpointLost`. Player must re-target.

10.3 Origin destroyed or recovered (non-KSC)

Scenario: Minmus base recovered for funds. **Behavior:** No vessels at origin → `Status = EndpointLost`. Vessels exist but empty → `Status = WaitingForResources`. Route persists. Resumes when resources appear. Surface origins have proximity fallback. KSC origins skip this check.

10.4 Destination tanks full

Scenario: Route delivers 200 LF. Base has 200/200 LF. **Behavior:** `Status = DestinationFull`. Cycle skipped -- no ghost replay, no origin deduction. Origin NOT deducted. Resumes when player uses fuel and capacity becomes available.

10.5 Destination partially full

Scenario: Base has room for 100 LF, full on Ox. Delivery: 150 LF + 183 Ox. **Behavior:** Per-resource independent: deliver 100 LF, 0 Ox. Origin pays full CostManifest (transit fuel was burned).

10.6 Player reverts past a dispatch

Scenario: Route dispatched at UT=50000. Player reverts to UT=49000. **Behavior:** Timeline events invalidated by epoch isolation. Origin resources restored via quicksave. Route state restored from .sfs.

10.7 Time warp past multiple cycles

Scenario: Three cycles due at UT=50000, 50500, 51000. **Behavior:** All processed sequentially. Each dispatch checks origin independently. First may deplete origin, blocking subsequent.

10.8 Transport still docked at recording end

Scenario: Player forgets to undock. **Behavior:** Validation fails. "Create Route" absent. Tooltip: "Transport must undock from destination."

10.9 No resources transferred during docking

Scenario: Player docks and undocks without transferring. **Behavior:** Validation fails. Tooltip: "No resource transfer detected during docking."

10.10 Multiple dock/undock in one recording chain

Scenario: Route recording chain has dock-transfer-undock-dock-transfer-undock. **Behavior:** Route analysis engine extracts each dock-transfer-undock sequence as a separate RouteStop. Delivery happens independently at each stop during chain-sequential playback.

10.11 Route dispatch while player is at destination

Scenario: Player at Mun base when delivery arrives. **Behavior:** Destination loaded. Uses `Part.RequestResource()`. Resources appear in real-time.

10.12 Competing routes at same origin

Scenario: Two routes share Minmus base. Base has enough for one, not both. **Behavior:** v1 — FIFO by NextDispatchUT. Future: player-configurable priority.

10.13 Linked route partner paused

Scenario: Route A and B linked. Player pauses B. **Behavior:** A dispatches on its own schedule. B resumes from its next scheduled dispatch when unpaused.

10.14 Recording deleted

Scenario: Source recording for a route is deleted. **Behavior:** Route orphaned — resource transfers continue, ghost replay absent. "No ghost" indicator in UI.

10.15 Save/load round-trip

Scenario: Save, load. **Behavior:** All Route fields serialized in ParsekScenario OnSave/OnLoad. State restored exactly.

---

11. v1 Limitations

- **Capacity changes during transit:** Delivery clamps to `maxAmount` at delivery time. Excess silently lost. Origin was already deducted at dispatch.
- **Zero transit duration:** Dispatch and delivery may process in same frame. Acceptable.
- **EC-only delivery:** Electric charge fluctuates rapidly. Status may flicker between Active and DestinationFull.
- **Resource not on destination:** If delivery manifest includes a resource the destination has no tanks for, that resource is silently skipped.
- **Origin loaded vs unloaded:** Same loaded/unloaded distinction as delivery.
- **Scene handling:** Route scheduler runs in all scenes via ParsekScenario. `FlightGlobals.Vessels` available for endpoint resolution.
- **Revert mechanism:** Route state serialized in .sfs. Quicksave load restores Route ConfigNode. Timeline events use epoch isolation.
- **Inventory delivery:** Inventory items delivered to destination cargo slots. If destination lacks available slots or the part type doesn't fit, excess items are silently skipped (same pattern as resource delivery clamping to maxAmount).
- **Crew delivery:** Generic kerbals (by trait, not named individuals) assigned to available seats at destination. Separate from the crew reservation system — route crew are generated at delivery time, not reserved in advance.
- **Route analysis edge cases:** The route analysis engine walks dock-transfer-undock sequences linearly. Complex docking patterns (dock to A, undock from A, dock to A again) are handled as separate stops at the same endpoint. Partial transfer detection (dock but no resource change) produces a stop with an empty delivery manifest, which is valid but does nothing.

---

12. What Doesn't Change

- **Recording system** — recordings unchanged. Cargo manifests (Phase 11: resources, inventory, crew) are additive metadata on Recording. The route module reads these fields but never writes to them.
- **Ghost playback engine** — no changes to GhostPlaybackEngine, IPlaybackTrajectory, IGhostPositioner. The route scheduler uses the same playback engine as loops, but sequences segments manually via `OnPlaybackCompleted` events rather than using the built-in loop toggle.
- **Loop system** — per-recording loop toggle, timing, cycle events all work as today. Routes do not use the loop system — they are siblings, not built on top of it. Both use the ghost playback engine, but through different scheduling paths.
- **Chain system** — chain segments, dock/undock boundaries, snapshots all unchanged.
- **Manifest capture systems (Phase 11)** — `ExtractResourceManifest`, `ComputeResourceDelta`, `DockTargetVesselPid` capture, and the three manifest types (resources, inventory, crew) are Phase 11

deliverables. They exist on Recording as additive fields, consumed read-only by both the route module and the UI tooltips.

- **Merge dialog** — route creation happens after commit via explicit "Start/End Route Recording" workflow, not during merge.
- **Crew reservation** — route crew delivery uses generic kerbals, separate from the existing crew reservation system. No integration with KerbalsModule needed.
- **Game actions system** — route events are new event types in existing ledger. No changes to recalculation engine.
- **Map markers** — deferred. No map view integration in v1.

---

13. Module Architecture

The logistics route system follows the same module pattern as the game actions system (v0.6): a self-contained directory with a thin orchestrator connecting it to Parsek's lifecycle hooks. The route module is removable by deleting the directory and removing the orchestrator calls — no behavioral changes to recording, playback, or the game actions system.

13.1 Directory structure

```
Source/Parsek/Logistics/
    Route.cs                    // data model (Route, RouteStop, RouteEndpoint, RouteStatus)
    RouteStore.cs               // static storage surviving scene changes (like
RecordingStore)
    RouteScheduler.cs           // dispatch/delivery evaluation + chain-sequential playback
(pure logic)
    RouteDelivery.cs            // ProtoPartResourceSnapshot modification on unloaded vessels
    RouteEndpointResolver.cs    // vessel finding by PID + surface proximity fallback
    RouteManifestComputer.cs    // derive delivery/cost manifests from recording chain
resources
    RouteAnalysisEngine.cs      // chain walk + stop extraction from committed recordings
    RouteOrchestrator.cs        // thin integration layer -- called from ParsekScenario hooks
```

13.2 Integration seams (4 total)

These are the only places where logistics code touches existing Parsek code:

Seam	Where	What	
Save/Load	`ParsekScenario.OnSave/OnLoad`	`RouteOrchestrator.OnSave(node)/OnLoad(node)`	N cl M R n ro
Scheduler tick	`ParsekScenario.Update`	`RouteOrchestrator.Tick(currentUT)`	Si n n ro

Playback completed	`ParsekPlaybackPolicy.HandlePlaybackCompleted`	`RouteOrchestrator.OnSegmentCompleted(evt)`	C th co tr b to ac ro o o
Timeline events	`Ledger` / `LedgerOrchestrator`	New `GameActionType` entries for ROUTE_DISPATCHED / ROUTE_DELIVERED	A en va d st

No changes to `FlightRecorder`, `GhostPlaybackEngine`, `RecordingStore`, `RecordingTree`, `ChainSegmentManager`, `RecordingOptimizer`, or any recording/playback code.

13.3 Read-only consumption of recording data

The route module is a read-only consumer of recording data. It reads:

- `rec.StartResources` / `rec.EndResources` -- resource manifests (Phase 11)
- `rec.DockTargetVesselPid` -- dock target vessel identification (Phase 11)
- `rec.StartBodyName`, `rec.StartLatitude`, `rec.StartLongitude` -- location context (Phase 10)
- Chain boundary trajectory points for dock-time coordinates

The route module never writes to Recording objects. It creates Route objects in its own `RouteStore`, serialized in its own `ROUTES` ConfigNode section inside `ParsekScenario`.

13.4 Resource modification path

Resource delivery modifies vessels that are not part of the recording system — they are real KSP vessels at endpoint locations:

```
RouteDelivery.DeliverResources(stopVessels, deliveryManifest)
    for each vessel:
        if loaded:  part.RequestResource(name, -amount)     // KSP API
        if unloaded: protoPartResource.amount += amount      // direct field write
```

This is completely independent of Parsek's recording/playback/ghost systems. No ghost, no recording, no trajectory — just a vessel and a number.

13.5 Lifecycle isolation

Routes have their own lifecycle, independent of recordings:

- **Creation:** from a committed recording chain (post-commit "End Route Recording" workflow), but the route is a separate entity with its own GUID.
- **Persistence:** own ConfigNode section (`ROUTES` in ParsekScenario), not part of recording metadata.

- **Deletion:** deleting a route does not affect its source recordings. Deleting a source recording orphans the route (no ghost replay, but resource transfers continue).
- **Revert:** route state is serialized in .sfs — quicksave/load restores it. Timeline events use the existing epoch isolation.

13.6 Why not a separate assembly now

The roadmap defers assembly extraction to the Gloops boundary (pre-Phase 13). For Phase 12, a directory-level module within `Parsek.csproj` is the right granularity. Routes need direct access to `Recording.StartResources` / `EndResources`, `RecordingStore.CommittedRecordings`, `Ledger`, and `ParsekScenario` lifecycle. Cross-assembly access would require making all of these `public` or adding an interface layer — friction without benefit. If Gloops extraction happens, routes stay in Parsek (they are Parsek policy, not ghost playback).

---

14. Backward Compatibility

- **Saves without routes:** Load fine. ROUTE ConfigNode absent.
- **Saves without resource manifests:** Load fine. Route creation unavailable for old recordings (no manifest data).
- **Old recordings:** Cannot become routes. Player must re-fly to create a recording with manifests.
- **Format:** All new data additive. No version bump. Missing nodes = no data.

---

15. Diagnostic Logging

15.1 Route creation

- `[Parsek][INFO][Route] Route created: id={id} name={name} recordings={count} stops={count} origin={body}({lat},{lon}) cost={manifest}`
- `[Parsek][INFO][Route] Route stop {n}: {body}({lat},{lon}) vesselPid={pid} delivery={manifest}`
- `[Parsek][INFO][Route] Route analysis: chain={count} recordings, found {count} stops, roundTrip={bool}`
- `[Parsek][INFO][Route] Route validation failed for chain: {reason}`

15.2 Dispatch evaluation

- `[Parsek][VERBOSE][Route] Dispatch check: route={name} nextUT={ut} currentUT={ut} status={status}`
- `[Parsek][INFO][Route] Dispatch: route={name} cycle={n} deducted={amounts} from origin at {body}({lat},{lon})`
- `[Parsek][INFO][Route] Dispatch delayed: route={name} — origin missing {resource}={needed} (available={have})`
- `[Parsek][INFO][Route] Dispatch skipped: route={name} — destination full (capacity={amounts})`
- `[Parsek][INFO][Route] Dispatch skipped: route={name} — linked route {linkedId} in transit`

15.3 Delivery

- `[Parsek][INFO][Route] Delivery: route={name} cycle={n} delivered={amounts} to {count} vessel(s) at {body}({lat},{lon})`

- `[Parsek][INFO][Route] Partial delivery: route={name} — {resource} delivered {actual}/{requested}`
- `[Parsek][WARN][Route] Delivery failed: route={name} — no vessels at destination`

15.4 Endpoint resolution

- `[Parsek][VERBOSE][Route] Endpoint resolve: {type} pid={pid} found={bool} fallback={proximity/none} vessels={count}`

15.5 State transitions

- `[Parsek][INFO][Route] Status change: route={name} {old} → {new}`

15.6 Timeline events

- `[Parsek][INFO][Route] Timeline event: ROUTE_DISPATCHED route={name} ut={ut}`
- `[Parsek][INFO][Route] Timeline event: ROUTE_DELIVERED route={name} ut={ut} amounts={amounts}`

---

16. Test Plan

16.1 Unit tests (pure logic, no Unity)

ExtractResourceManifest

- Empty ConfigNode → empty manifest. *Catches: null deref on missing PART nodes.*
- Single part, one resource → manifest with one entry. *Catches: parsing errors.*
- Multi-part, same resource → amounts summed. *Catches: overwrite instead of accumulate.*
- Multiple resource types → all present. *Catches: single-resource assumption.*
- Missing RESOURCE node → skipped gracefully. *Catches: null deref.*
- Zero-amount resource → included (maxAmount matters). *Catches: filtering out zeros.*

ComputeDeliveryManifest

- Normal transfer: pre-dock 200 LF, pre-undock 50 LF → delivery 150 LF. *Catches: wrong snapshot pair.*
- Resource increased (EC): pre-dock 50, pre-undock 180 → delivery 0. *Catches: including increases.*
- Mixed: some decrease, some increase → only decreases. *Catches: all-or-nothing.*
- No decreases → empty manifest (validation rejects). *Catches: false positive.*

Route analysis engine

- Single dock-transfer-undock → one stop extracted. *Catches: no stop found.*
- Two dock-transfer-undock sequences → two stops. *Catches: only finding first/last.*
- Dock without undock → validation fails. *Catches: incomplete pair.*
- No dock events → validation fails. *Catches: false acceptance.*
- Dock with no resource change → stop with empty delivery manifest. *Catches: crash on empty delta.*
- Resource pickup (negative delta) → negative delivery in manifest. *Catches: filtering negatives.*
- Round-trip detection (end within 500m of origin) → isRoundTrip true. *Catches: missing proximity check.*
- Multi-body chain (Kerbin origin, Mun stops) → correct body per stop. *Catches: assuming same body.*

Route validation

- Dock + transfer + undock → valid. *Catches: false rejection.*
- Dock + undock, no transfer → invalid. *Catches: false acceptance.*
- Dock + transfer, no undock → invalid. *Catches: missing undock check.*
- No dock → invalid. *Catches: missing dock check.*

Chain-sequential playback

- Segment 0 completes → scheduler starts segment 1. *Catches: stuck on first segment.*
- Last segment completes → cycle count incremented, status returns to Active. *Catches: stuck InTransit.*
- Stop between segments → delivery triggered. *Catches: missed delivery.*
- No stop between segments → no delivery, next segment starts. *Catches: false delivery.*

Dispatch evaluation

- KSC origin, capacity available → dispatch, no deduction. *Catches: deducting from KSC.*
- Non-KSC, sufficient → dispatch, deducted. *Catches: skipping deduction.*
- Non-KSC, insufficient → delayed. *Catches: dispatching without resources.*
- Destination full → skipped, origin NOT deducted. *Catches: wasted deduction.*
- Partial capacity → per-resource independent, full cost. *Catches: coupled delivery.*
- Linked partner in transit → skipped. *Catches: ignoring link.*

Synodic period

- Kerbin-Mun → ~6.4 days. *Catches: formula error.*
- Mun-Laythe → walks to Kerbin/Jool. *Catches: cross-system hierarchy.*
- Same body → 0. *Catches: division by zero.*
- Sun-orbiting → 0. *Catches: missing guard.*
- Equal periods → 0. *Catches: T1==T2 division.*

Endpoint resolution

- PID exists → found. *Catches: skipping PID.*
- PID gone, surface, vessel within 50m → fallback. *Catches: missing fallback.*
- PID gone, surface, nothing nearby → empty. *Catches: false match.*
- PID gone, orbital → empty. *Catches: orbital fallback.*

16.2 Log assertion tests

- Route creation logs manifest and endpoints. *Catches: silent creation.*
- Dispatch delayed logs missing resource and amounts. *Catches: silent delay.*
- Delivery logs amounts per vessel. *Catches: silent delivery.*
- Status change logs old→new. *Catches: silent transition.*
- Partial delivery logs reason. *Catches: silent partial.*

16.3 Serialization round-trip tests

- Route serialize → deserialize → all fields match. *Catches: missing field.*
- Multi-stop route round-trip → all stops preserved with correct order. *Catches: stop ordering lost.*
- RecordingIds list round-trip → all IDs preserved in order. *Catches: chain ordering lost.*
- ResourceManifest round-trip with full precision. *Catches: locale formatting.*
- Null LinkedRouteId survives round-trip. *Catches: empty vs null.*
- In-transit route with PendingDeliveryUT and CurrentSegmentIndex survives. *Catches: transit state lost.*

16.4 Integration tests (synthetic recordings)

- Recording chain with dock+transfer+undock → route analysis extracts one stop. *Catches: chain structure mismatch.*
- Recording chain with two dock-transfer-undock pairs → two stops extracted. *Catches: multi-stop analysis failure.*
- Recording chain without undock → validation rejects. *Catches: missing chain event check.*
- Inject route with multi-stop into save → loads correctly with all stops. *Catches: ParsekScenario integration.*

---

17. Open Questions (deferred to v1.1+)

- **Map view integration:** Route lines on the map. Deferred.
- **Dispatch priority for competing routes:** v1 uses FIFO by NextDispatchUT.

Appendix A: Gameplay Scenarios (from Step 2)

A.1 Fuel Delivery Rover

Base with empty fuel tank near KSC. Rover drives to base, docks, transfers 150 LF, undocks, drives back. Route: 150 LF per cycle, KSC origin (free), interval = recording duration.

A.2 Orbital Monoprop Resupply

Kerbin station at 100km. Capsule from KSC: launch, dock, transfer 650 MP, undock, deorbit. Ghost visible during launch and station approach (RELATIVE frame). Transit invisible.

A.3 Minmus Ore Delivery (chained routes)

Route A: KSC → Minmus base (fuel). Route B: Minmus base → Kerbin depot (ore). B is gated by resource availability at Minmus. A feeds B.

A.4 Eeloo Supply Run (interplanetary)

Dispatch at Kerbin-Eeloo synodic period (~1.9 years). Gravity assists use same two-body approximation. Player can override.

A.5 Failure Cases

Destination destroyed (surface: proximity reconnects; orbital: route pauses). Origin empty (delayed). Destination full (skipped, no deduction). Revert (epoch isolation). Time warp (sequential processing). Transport still docked (validation rejects).

Appendix B: Reference Documents

- `docs/dev/plans/phase-11-resource-snapshots.md` — Phase 11 detailed plan (resource snapshots, module architecture, route recording workflow)
- `docs/dev/research/logistics-network-design.md` — logistics network research
- `docs/dev/research/loop-playback-and-logistics.md` — loop mechanics and orbital drift
- `docs/dev/research/resource-snapshots-preparation.md` — infrastructure analysis and unloaded vessel modification
- `docs/mods-references/` — background resource processing patterns from other KSP mods
- `docs/roadmap.md` — Phase 11, 11.5, 12